

**POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY**

COURSE ASSIGNMENT

Student Information:

Name: Bui Quang Anh

Student ID: B22DCDT008

Class: E22CNPM-01

Lecturer:

Tran Dinh Que

Hanoi, June 11, 2026

Contents

1	From Monolithic to Microservices and DDD	3
1.1	From Monolithic to Microservices	3
1.1.1	What is Monolithic Architecture?	3
1.1.2	What is Microservices Architecture?	3
1.2	Microservices Design with Domain-Driven Design (DDD)	4
1.2.1	What is Domain-Driven Design?	4
1.2.2	Microservices Design Process with DDD	5
1.3	Case Study: Healthcare System	6
1.3.1	Phase 1: Domain Analysis & Subdomain Classification	6
1.3.2	Asynchronous Communication (REST Callback)	6
1.3.3	Phase 2: Defining Bounded Contexts	6
1.3.4	Phase 3: Mapping to Microservices	7
1.3.5	Custom API Gateway Pattern	7
1.3.6	Phase 4: Tactical Design Example	7
1.3.7	Phase 5: Context Mapping and Service Interaction	7
1.3.8	Figures: Architecture Comparison and Healthcare Decomposition	8
1.3.9	Quantitative Comparison: Monolith vs Microservices	9
2	E-commerce System and Domain-Driven Design	10
2.1	Requirements Definition	10
2.2	System Decomposition using DDD	10
2.2.1	Group I: User Service	11
2.2.2	Group II: Product Service	11
2.2.3	Group III: Order Service	12
2.2.4	Group IV: AI Service	12
	Detailed Analysis and Service Design	12
2.2.5	Analytical Charts	14
2.2.6	Service Decomposition Diagram	15
2.2.7	Class Diagrams and Data Models	17
2.2.8	Upgraded UML Diagrams	17

2.2.9	Schema Specification (per service)	18
	AI Pipeline: End-to-end (Supplement)	23
2.3	System Design	26
2.3.1	Individual Service Design	26
2.3.2	Service Isolation Strategy	26
2.3.3	Database Schema and App Structure	27
2.3.4	Connect Services (Inter-service Communication)	28
3	AI Service	31
3.1	Architecture Overview	31
3.2	Module 1: Sequence Behavior Prediction	31
3.2.1	Dataset and Problem Formulation	31
3.2.2	Model Architecture	32
3.2.3	Training Result and Model Selection	32
3.3	Module 2: Knowledge Base Graph with Neo4j	33
3.3.1	Graph Construction	33
3.3.2	Cypher Query Example	34
3.4	Module 3: GraphRAG Chat and E-commerce Integration	34
3.4.1	GraphRAG Query Pattern	34
3.4.2	System Integration	34
3.4.3	AI Components Visual Summary	35
4	Code Implementation and Integration	36
4.1	Project Structure	36
4.2	Group II: Product Service Implementation	37
4.2.1	Django Models	37
4.2.2	DRF Serializers	38
4.2.3	ViewSetS with Stock Reduction	38
4.3	AI Service: Current Implementation	39
4.3.1	Recommendation API (NCF)	39
4.3.2	RAG Chat API (FAISS + Sentence-Transformers)	39
4.3.3	Sequence and Graph Extension Modules	39
4.4	Docker Compose Orchestration	40
4.5	Chapter 5 — Demo and Code Runbook	43
4.5.1	Run and Demo Steps	43
4.5.2	Code Highlights (paths)	43
4.5.4	Minimal Example: Calling Recommendation API	44
4.6	Verification Results	44
	Conclusion	45

Chapter 1

From Monolithic to Microservices and DDD

1.1 From Monolithic to Microservices

1.1.1 What is Monolithic Architecture?

A monolithic architecture is a traditional model for designing software applications. In this approach, the entire application is constructed as a single, unified, and indivisible unit. All components, including the user interface, business logic, data access layer, and background processing, are tightly integrated into a single codebase and deployed as a single executable or artifact (e.g., a WAR or JAR file in Java).

While a monolithic architecture is relatively simple to develop, test, and deploy in the early stages of a project, it presents significant challenges as the system grows. Key limitations include:

- **Scalability Issues:** To scale a specific module under heavy load, the entire application must be replicated, leading to inefficient resource utilization.
- **Tight Coupling:** Changes in one module can inadvertently impact others, increasing the risk of widespread failures.
- **Technology Lock-in:** It is extremely difficult to adopt new technologies or frameworks for specific components, as the entire monolith is bound to a single technology stack.

1.1.2 What is Microservices Architecture?

Microservices architecture is a modern architectural style that structures an application as a collection of small, autonomous, and loosely coupled services. Each microservice is

built around a specific business capability, runs in its own process, and communicates with other services through lightweight mechanisms, such as HTTP/REST or message brokers (e.g., RabbitMQ, Kafka).

This approach addresses the shortcomings of monolithic applications by offering several theoretical advantages:

- **Independent Deployment:** Services can be updated and deployed independently without affecting the rest of the system.
- **Targeted Scalability:** Individual services can be scaled horizontally based on their specific workload demands.
- **Polyglot Persistence and Programming:** Different services can be implemented using the most appropriate programming languages and database technologies for their specific tasks.
- **Fault Isolation:** A failure in one microservice (e.g., a memory leak) does not bring down the entire application.

1.2 Microservices Design with Domain-Driven Design (DDD)

1.2.1 What is Domain-Driven Design?

Domain-Driven Design (DDD) is a software engineering methodology introduced by Eric Evans. It focuses on modeling software to match a complex domain based on input from domain experts. In the context of microservices, DDD provides the theoretical framework and strategic patterns required to define clear and logical boundaries for each service, ensuring that they are truly cohesive and loosely coupled.

At the core of DDD are several strategic patterns:

- **Ubiquitous Language:** A common, rigorous language shared by both technical teams and business stakeholders to describe the domain model without ambiguity. This ensures that the code directly reflects the business reality.
- **Subdomains:** Complex domains are broken down into smaller, manageable subdomains. These are categorized into *Core Domains* (the primary business differentiator), *Supporting Domains* (necessary but not the core business), and *Generic Domains* (off-the-shelf solutions like accounting or authentication).
- **Bounded Context:** A clear linguistic and conceptual boundary within which a specific domain model is strictly applicable. In architectural terms, a Bounded

Context often perfectly aligns with the boundary of a microservice, ensuring that data and logic are encapsulated and protected from outside interference.

1.2.2 Microservices Design Process with DDD

The process of decomposing a system into microservices using DDD involves both strategic and tactical design steps:

1. **Understand the Core Domain & Subdomains:** Collaborate with domain experts to map the entire business landscape. Identify what brings the most value (Core Domain) and what are auxiliary functions (Supporting/Generic Domains).
2. **Define the Ubiquitous Language:** Establish a unified vocabulary to ensure that all team members use the same terminology for business concepts, eliminating translation errors between business and IT.
3. **Identify Bounded Contexts:** Define the boundaries where a specific Ubiquitous Language applies. For example, a "Product" in an Inventory context has different attributes (weight, dimensions) compared to a "Product" in a Sales context (price, marketing description).
4. **Connect Services (Inter-service Communication)**
5. **Define Context Maps:** Map out how different Bounded Contexts interact with each other to establish integration contracts. Common mapping patterns include:
 - **Synchronous Communication (REST API)**
 - **Anti-Corruption Layer (ACL):** A translation layer that prevents a downstream context's model from being polluted by an upstream context's model.
 - **Shared Kernel:** A small, explicitly shared part of the model between two contexts.
 - **Open Host Service:** Defining a protocol or API that gives access to your subsystem as a set of services.
6. **Tactical Design (Inside the Boundary):** Within each microservice (Bounded Context), apply tactical DDD patterns to structure the complex business logic:
 - **Entities:** Objects defined by an identity thread that runs through time (e.g., a User with an ID).
 - **Value Objects:** Immutable objects that describe characteristics or attributes but have no conceptual identity (e.g., an Address or a Currency).

- **Aggregates:** Clusters of domain objects that can be treated as a single unit for data changes, ensuring transactional consistency.
- **Repositories:** Mechanisms that encapsulate storage, retrieval, and search behavior to emulate a collection of objects.

1.3 Case Study: Healthcare System

To demonstrate the practical application of microservices analysis and design using DDD, we examine a Healthcare System. In a traditional setting, hospital operations—ranging from patient registration to doctor scheduling and billing—are managed by a massive monolithic application. As the hospital grows, deploying new features (like telemedicine integration) becomes risky and slow. We will apply the DDD process to decompose this system into microservices.

1.3.1 Phase 1: Domain Analysis & Subdomain Classification

First, we analyze the hospital's operations to identify subdomains:

1.3.2 Asynchronous Communication (REST Callback)

- **Core Domain:** Clinical Care (Diagnosis, Treatment, Medical Records). This is where the hospital provides its primary value.
- **Supporting Domain:** Scheduling (Doctor availability, Patient appointments) and Pharmacy (Inventory management).
- **Generic Domain:** Billing and Invoicing (Standard financial operations).

1.3.3 Phase 2: Defining Bounded Contexts

Based on the subdomains and the Ubiquitous Language, we define distinct Bounded Contexts to serve as boundaries for our microservices:

- **Clinical Context:** Focuses on patient health, medical history, and treatment plans. The term "Patient" here refers to a biological entity receiving care.
- **Scheduling Context:** Focuses on time management and resource allocation. The term "Patient" here merely refers to a time-slot occupant.
- **Billing Context:** Focuses on financial transactions. The term "Patient" here translates to an "Account Holder" or "Payer".

Notice how the term "Patient" has different meanings and attributes depending on the context. This justifies the separation into different Bounded Contexts.

1.3.4 Phase 3: Mapping to Microservices

The Bounded Contexts map directly to individual microservices, each with its own independent database:

- **Patient Record Service:** Manages the Clinical Context.

1.3.5 Custom API Gateway Pattern

- **Appointment Service:** Manages the Scheduling Context.
- **Billing Service:** Manages the Billing Context.

1.3.6 Phase 4: Tactical Design Example

Inside the **Patient Record Service**, we apply tactical design. We define an **Aggregate** called **PatientRecord**. The **Patient** entity is the **Aggregate Root**. It contains **Value Objects** like **BloodPressure** or **AllergyInfo**. Any update to a patient's medical history must go through the **Patient** Aggregate Root to ensure business rules (e.g., preventing incompatible medications) are enforced transactionally.

1.3.7 Phase 5: Context Mapping and Service Interaction

To maintain autonomy, these microservices must communicate loosely, typically via asynchronous messaging (e.g., using RabbitMQ or Kafka), acting as an **Anti-Corruption Layer**:

- When a doctor completes a checkup, the **Appointment Service** publishes a Domain Event: **AppointmentCompletedEvent**.
- The **Billing Service** subscribes to this event. Upon receiving it, it translates the appointment details into its own domain model (an **Invoice**) and processes the payment.
- This event-driven architecture ensures that if the Billing Service crashes, the Clinical and Scheduling services remain fully operational, achieving high fault isolation.

1.3.8 Figures: Architecture Comparison and Healthcare Decomposition

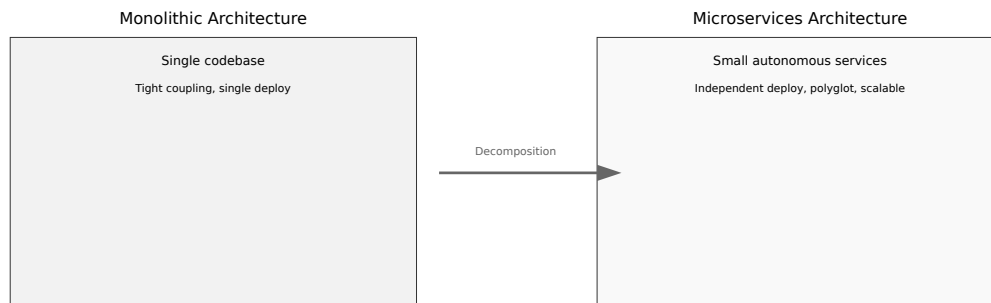


Figure 1.1: Comparison: Monolithic vs Microservices (high level).

This figure contrasts the operational and deployment differences between monolithic and microservices architectures: independent deployment units, separate databases per service, and typical communication patterns (REST or messaging) used to integrate bounded contexts.

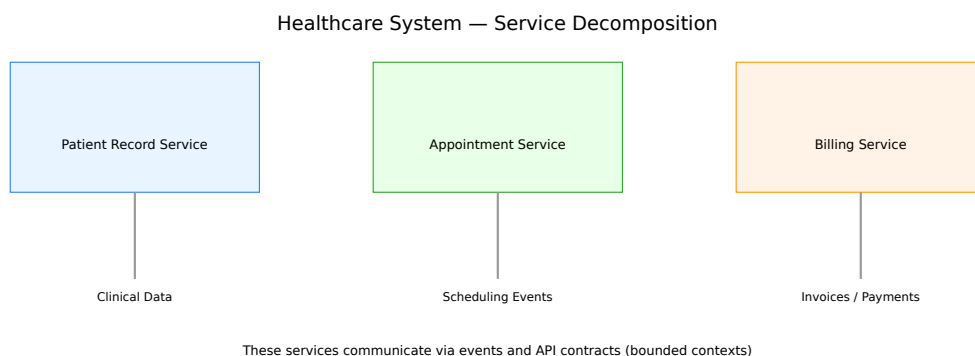


Figure 1.2: Healthcare Case Study — service decomposition diagram.

The decomposition diagram shows how clinical, scheduling, and billing bounded contexts map to separate services, highlighting data ownership and where asynchronous events or anti-corruption layers are appropriate.

1.3.9 Quantitative Comparison: Monolith vs Microservices

To make the architectural differences more analytical (not only illustrative), Table 1.1 summarizes a small set of measurable attributes comparing a typical Monolithic deployment with a Microservices deployment. Replace the example values with measured or estimated values where possible.

Table 1.1: Quantitative comparison between Monolithic and Microservices

Metric	Monolithic (example)	Microservices (example)	Notes
Typical deployment time (minutes)	10–30	1–5 (per service)	Smaller artifact per service; overall rollout may need orchestration
Scale unit	whole application	individual service instance	Finer-grained scaling with possible higher infra cost
Fault blast radius	entire application	single service (bounded)	Depends on isolation, retries, and circuit-breakers
Team ownership granularity	coarse	per-service teams	Easier autonomous changes in microservices
Operational overhead	low–medium	medium–high	More monitoring, CI/CD, service discovery required
Mean time to recovery	minutes–hours	minutes (service-level)	Assumes automated restarts and health checks

Chapter 2

E-commerce System and Domain-Driven Design

2.1 Requirements Definition

A modern E-commerce platform is a complex software ecosystem designed to facilitate online transactions seamlessly while providing an engaging user experience. The core functionalities required for such a system include:

- **User Management:** Handling registration, authentication (JWT), authorization, and role-based access control for Administrators, Staff, and Customers.
- **Product Catalog Management:** Organizing, storing, and retrieving product information, including categories, descriptions, pricing, and stock levels. Staff can perform CRUD operations on the catalog.
- **Order Processing:** Enabling customers to add items to a shopping cart, place orders, and track order lifecycle (Pending → Completed/Cancelled). Stock is automatically deducted upon order creation.
- **AI-Powered Intelligence:** Analyzing customer behavior, providing personalized product recommendations using Deep Learning, and offering a smart shopping assistant powered by RAG (Retrieval-Augmented Generation).

2.2 System Decomposition using DDD

To handle the complexity of the platform and to prepare for a microservices architecture, we apply Domain-Driven Design to decompose the system into distinct Bounded Contexts. Each context encapsulates specific business operations and data, reducing coupling across the entire platform. The breakdown yields four groups of microservices:

2.2.1 Group I: User Service

This bounded context is responsible for identity and access management. It handles all operations related to user data, ensuring security and proper access control.

- **Responsibilities:** Managing roles including Admin, Staff, and Customer.
- **Scope:** Registration, login (using JWT), profile updates, and role-based access control.

2.2.2 Group II: Product Service

This bounded context manages the core inventory of the platform. It serves product catalogs to customers and provides an administrative interface for staff to update the catalog.

- **Responsibilities:** Managing Products and Categories.
- **Requirement:** The system seeds an initial catalog of 12 categories with 18 products covering the Tech & Electronics theme:**Nexus Tech Hub**. The categories and their seeded products are:
 - 1. **Laptops** — Apple MacBook Pro 14 M3, Dell XPS 15, Lenovo ThinkPad X1 Carbon
 - 2. **Smartphones** — Apple iPhone 15 Pro Max, Samsung Galaxy S24 Ultra, Google Pixel 8 Pro
 - 3. **Tablets** — iPad Pro
 - 4. **Smartwatches** — Apple Watch Series 9
 - 5. **Audio & Headphones** — Sony WH-1000XM5, AirPods Pro 2nd Gen
 - 6. **Cameras & Photography** — Sony A7 IV
 - 7. **PC Components** — NVIDIA RTX 4090
 - 8. **Networking Devices** — ASUS ROG Router
 - 9. **Peripherals & Accessories** — Keychron Q1 Pro, Logitech MX Master 3S
 - 10. **Gaming Consoles** — PlayStation 5
 - 11. **Smart Home Devices** — Google Nest Hub
 - 12. **Monitors & Displays** — Samsung Odyssey G9

2.2.3 Group III: Order Service

This bounded context encapsulates the core business logic of purchasing. It manages the full order lifecycle from creation to completion.

- **Responsibilities:** Managing Orders and OrderItems. Staff can update order status (Pending → Completed/Cancelled).
- **Scope:** Order creation with stock validation, automatic stock deduction via REST callback to Product Service, order history for customers, and status management for staff.

2.2.4 Group IV: AI Service

This bounded context provides artificial intelligence capabilities to enhance the shopping experience.

- **Responsibilities:** Sequence behavior prediction, graph knowledge-base construction, and GraphRAG-style conversational recommendations.
- **Technology:** Built with FastAPI (instead of Django) to demonstrate polyglot microservices. Uses PyTorch (RNN/LSTM/biLSTM), Neo4j + Cypher for graph reasoning, and supporting analytics/RAG modules.
- **Data Source:** Generated sequential dataset `data_user500.csv` (500 users, 8 actions/user) and behavioral logs from the running platform.

Detailed Analysis and Service Design (Supplement)

Captured Frontend Evidence

.

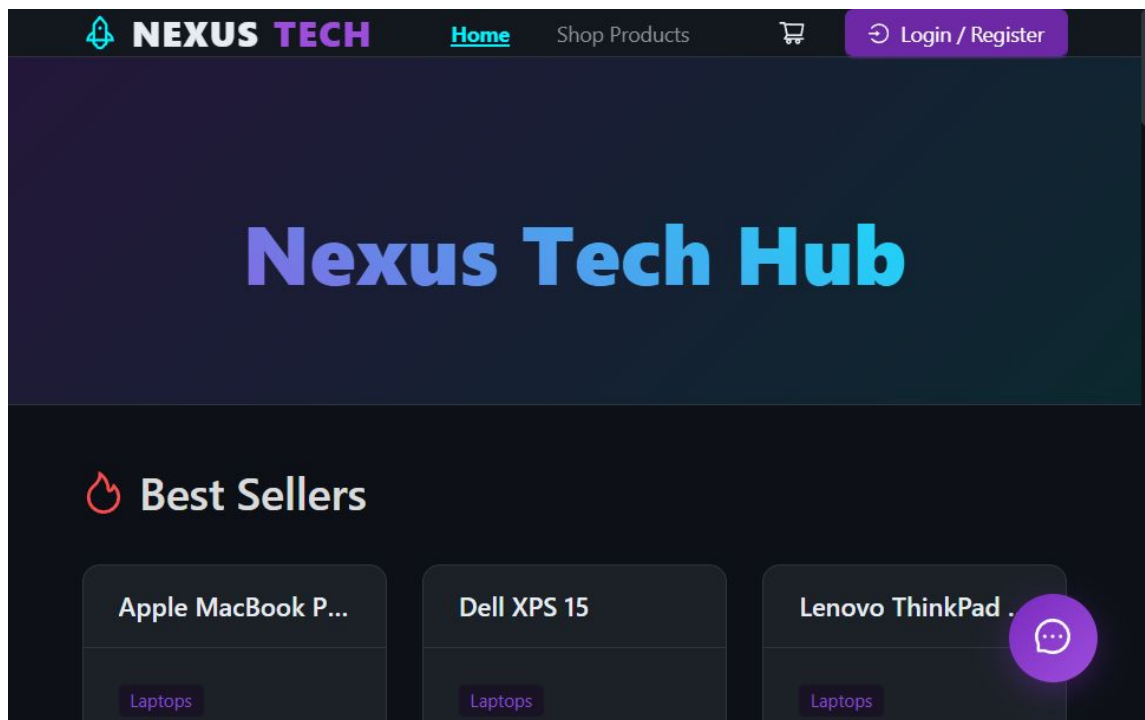


Figure 2.1: Frontend home page (Nexus Tech Hub storefront).

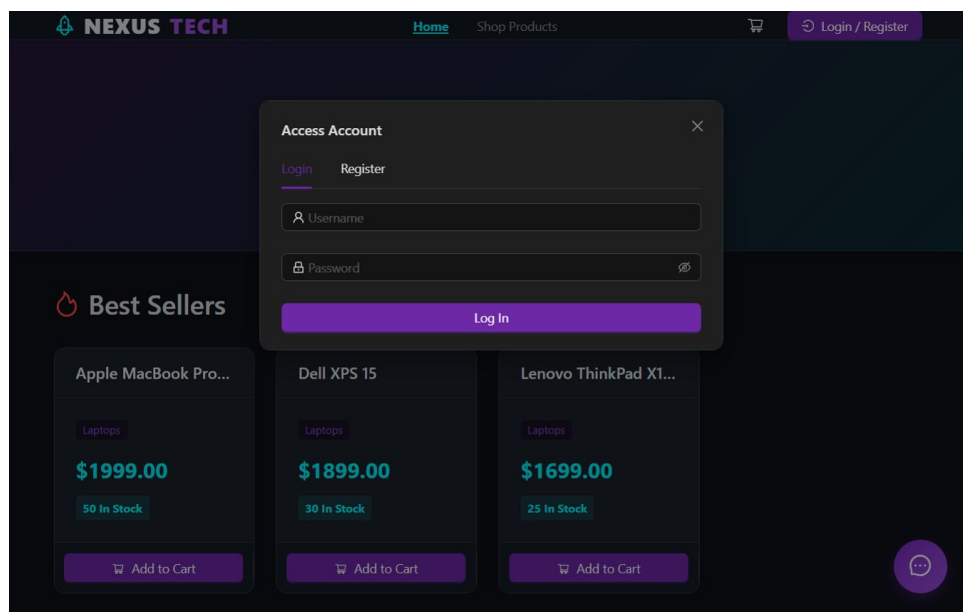


Figure 2.2: Authentication modal (Login/Register).

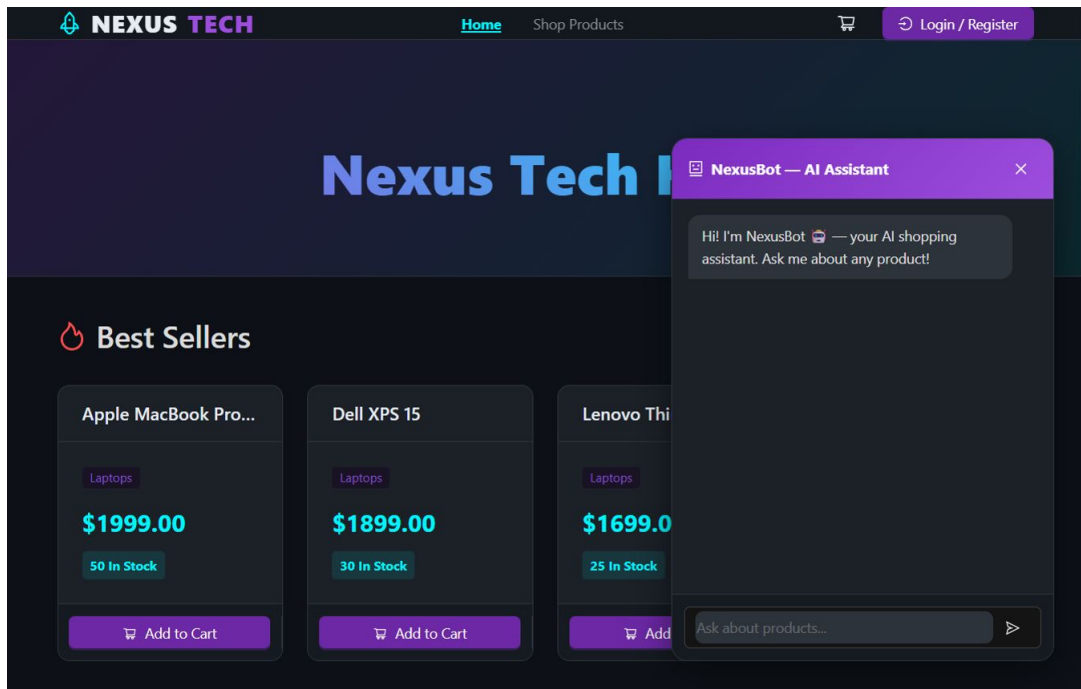


Figure 2.3: Customer AI assistant chat widget (NexusBot).

2.2.5 Analytical Charts

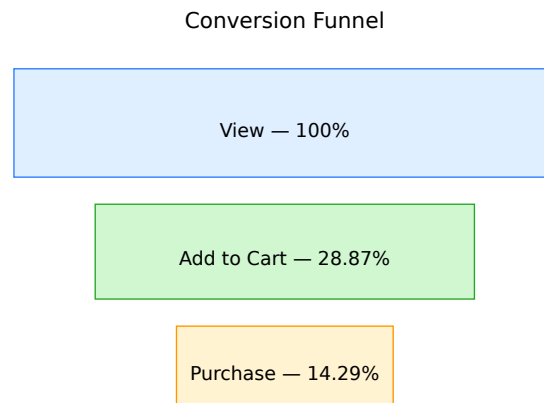


Figure 2.4: Conversion funnel and key ratios used in requirement analysis.

The conversion funnel illustrates user journey stages (View $\rightarrow$ Cart $\rightarrow$ Purchase) used to derive key metrics for recommendation prioritization and A/B testing targets.

Top Viewed Products (sample)

Prod A Prod B Prod C Prod D Prod E

Figure 2.5: Top viewed and top purchased product distributions.

This chart highlights product popularity signals (views vs purchases) used by the recommendation pipeline to weight candidate ranking and sampling strategies.

2.2.6 Service Decomposition Diagram

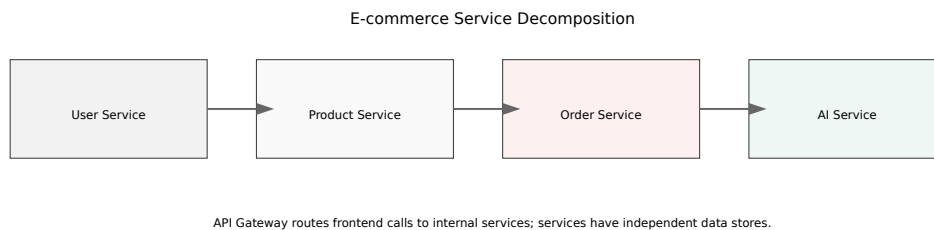


Figure 2.6: High-level service decomposition and boundaries.

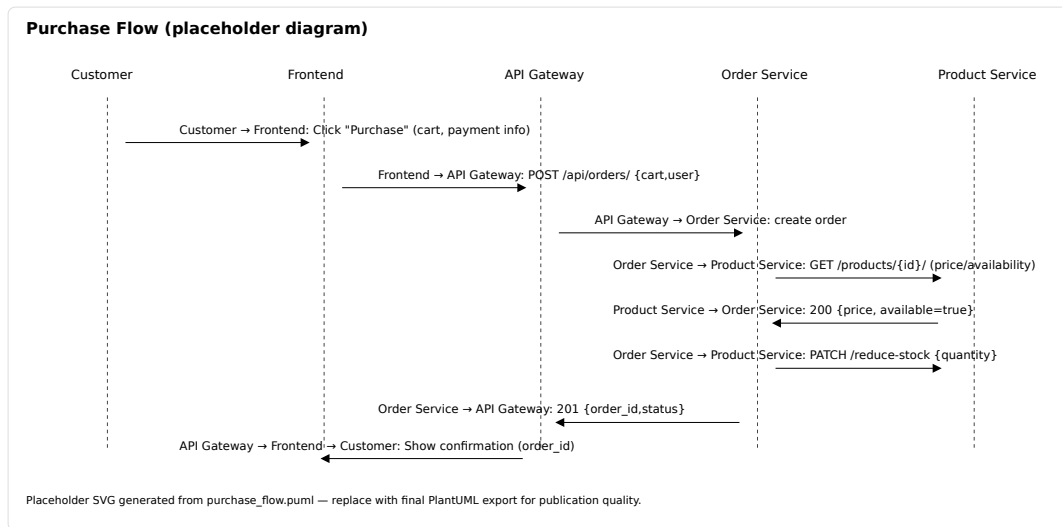


Figure 2.7: Purchase flow sequence diagram (Frontend → Gateway → Product → Order → Stock).

Figure above shows a simplified sequence for an end-to-end purchase request; replace with high-fidelity UML export if available.

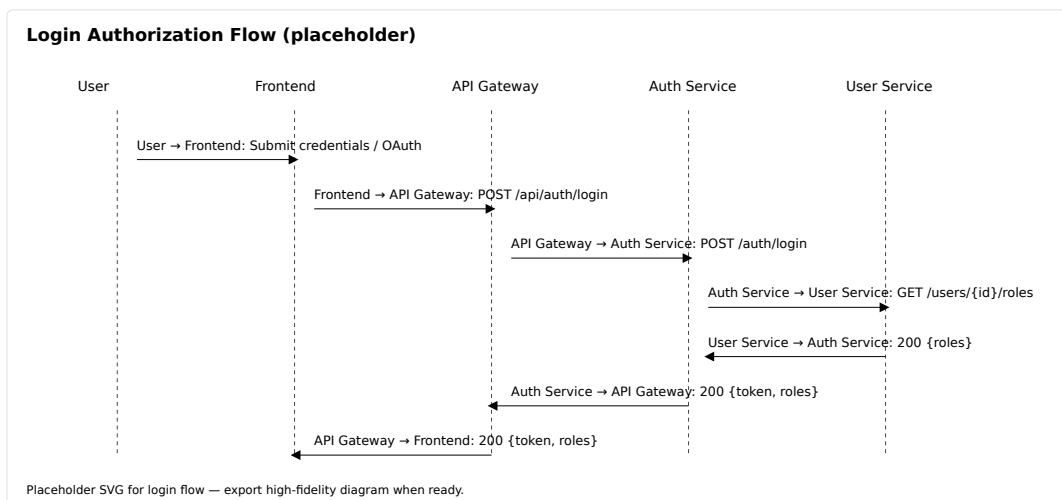


Figure 2.8: Login and authorization flow (Admin / Staff / Customer).

The login flow above outlines credential exchange, token issuance, and role-based authorization at the gateway. The service decomposition visualizes the runtime topology used in this project: one Django-based service per core domain, a FastAPI-based AI service, an API gateway proxy, databases, and the frontend; arrows indicate common synchronous interactions.

2.2.7 Class Diagrams and Data Models

User Service — Class Sketch

Class: User

- id: Integer
- username: String
- email: String
- password_hash: String
- role: Enum{ADMIN,STAFF,CUSTOMER}

Derived table: users_user(id PK, username, email, password_hash, role, is_active, cre

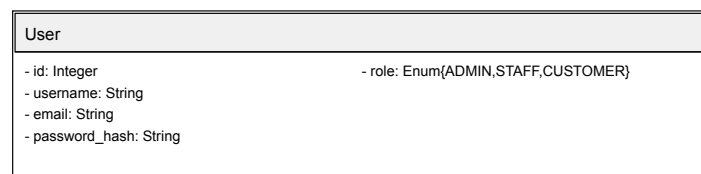
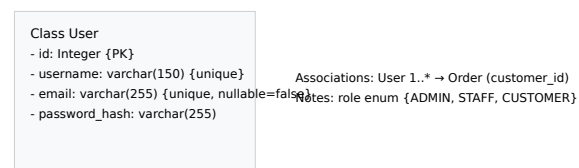


Figure 2.9: UML sketch for the `User` entity and primary attributes.

The UML sketch maps the domain attributes to the persisted `users_user` table described above.

2.2.8 Upgraded UML Diagrams

User Service — UML (placeholder)



Placeholder SVG — export proper UML (PlantUML) for publication.

Figure 2.10: User Service — class diagram (shows PK, unique constraints, and associations).

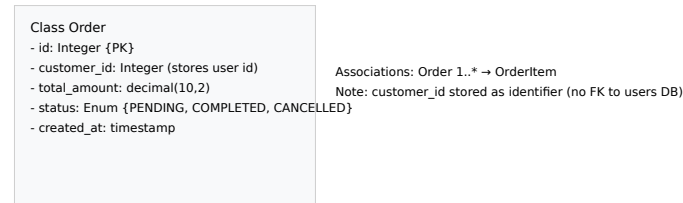
Product Service — UML (placeholder)



Placeholder SVG — replace with final UML export.

Figure 2.11: Product Service — class diagram (category relation, indexes).

Order Service — UML (placeholder)



Placeholder SVG — refine to show multiplicity and FK mapping.

Figure 2.12: Order Service — class diagram (Order, OrderItem multiplicities).

2.2.9 Schema Specification (per service)

For examiners who ask about DB-level details, each service includes a concise schema specification listing PKs, unique keys, indexes, nullability, and on-delete behaviors.

User Service — Schema Specification

Column	Type / Constraints	Notes (indexes / on-delete)
id	Integer (PK, auto)	primary key
username	varchar(150) NOT NULL UNIQUE	idx_username, unique
email	varchar(255) NOT NULL UNIQUE	idx_email, unique
password _{hash}	varchar(255) NOT NULL	—
role	Enum ADMIN, STAFF, CUSTOMER NOT NULL	default=CUSTOMER
is _{active}	boolean NOT NULL DEFAULT true	—
created _{at}	timestamp NOT NULL DEFAULT now()	—

Product Service — Schema Specification

Column	Type / Constraints	Notes (indexes / on-delete)
id	Integer (PK, auto)	primary key
category _{<i>i</i>} <i>d</i>	Integer NOT NULL	FK → categories(id); ON DELETE RESTRICT
name	varchar(255) NOT NULL	idx _{<i>p</i>} <i>product_name</i>
description	text	—
price	decimal(10,2) NOT NULL	—
stock	integer NOT NULL DEFAULT 0	idx _{<i>s</i>} <i>stock(foravailabilityqueries)</i>

Order Service — Schema Specification

Column	Type / Constraints	Notes (indexes / on-delete)
id	Integer (PK, auto)	primary key
customer _{<i>i</i>} <i>d</i>	Integer NOT NULL	stored as identifier; no cross-service FK
total _{<i>a</i>} <i>mount</i>	decimal(10,2) NOT NULL	—
status	EnumPENDING,COMPLETED,CANCELLED NOT NULL	index on status
created _{<i>a</i>} <i>t</i>	timestamp NOT NULL DEFAULT now()	—

Product Service — Class Sketch

Class: Category

- id
- name
- description

Class: Product

- id
- category_id (FK)
- name
- price
- stock

- description

Derived tables: products_category, products_product

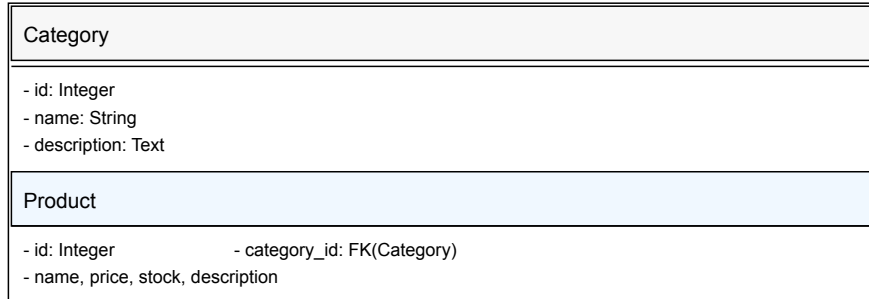


Figure 2.13: UML sketch for Category and Product models.

The diagram shows the Category to Product relationship and key persisted attributes used by the RAG knowledge base.

Order Service — Class Sketch

Class: Order

- id
- customer_id
- status
- total_amount

Class: OrderItem

- id
- order_id (FK)
- product_id
- quantity
- price

Derived tables: orders_order, orders_orderitem

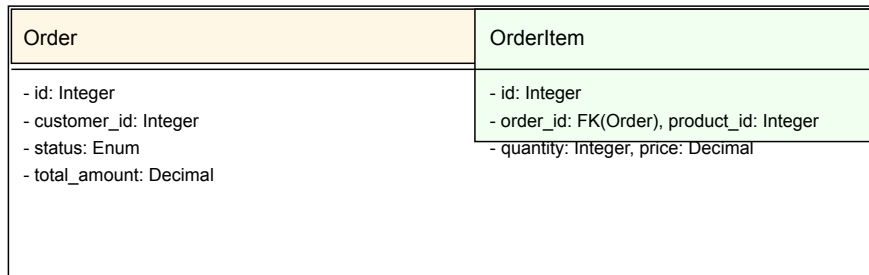


Figure 2.14: UML sketch for `Order` and `OrderItem` aggregates.

The `Order` aggregate contains `OrderItem` entries; note that `customer_id` and `product_id` are stored as identifiers to preserve service autonomy.

AI Service — Data Artifacts

Files / artifacts:

- `data/events.csv` (kaggle-like events)
- `data/data_user500.csv` (generated sequences)
- `models/model_best.pt` (sequence model)
- `data/faiss_index/products.index` + `products.json` (RAG KB)
- Neo4j graph (nodes: `User`, `Product`; relationships: `VIEW`, `CLICK`, `ADD_TO_CART`)

Captured Neo4j Evidence

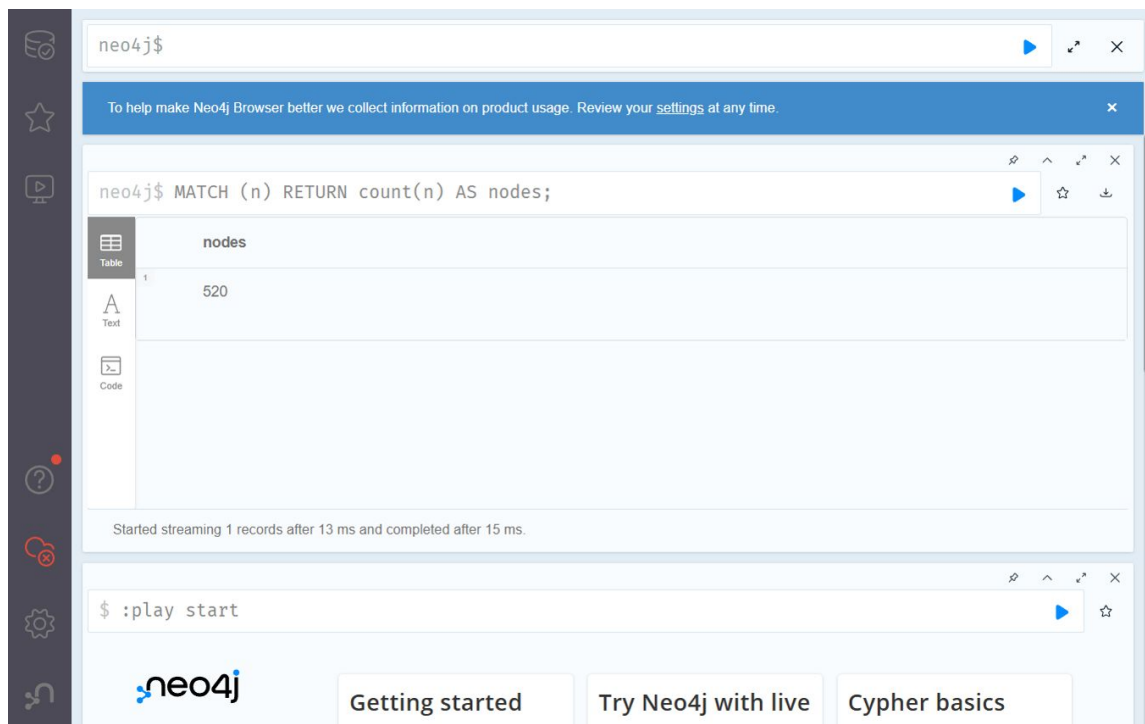


Figure 2.15: Neo4j Browser query result summary (node count verification).

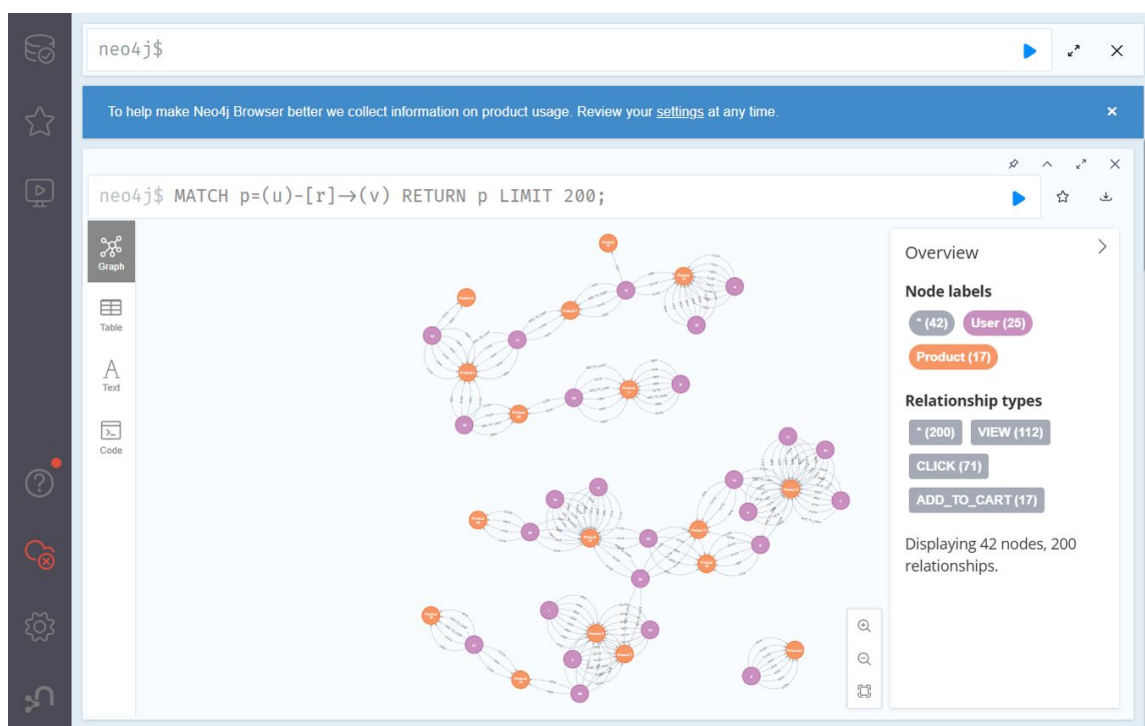


Figure 2.16: Neo4j Graph view with dense User-Product interaction relationships.

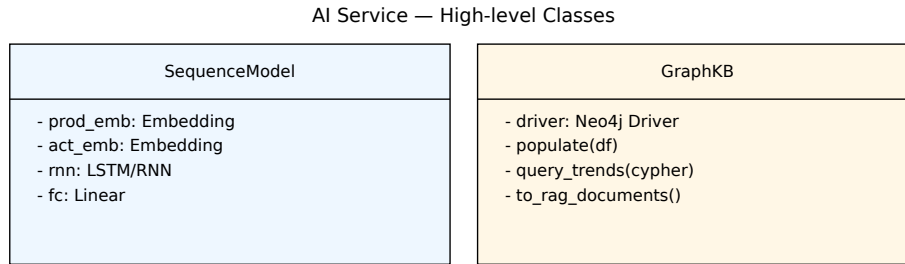


Figure 2.17: High-level AI service class sketch: SequenceModel and GraphKB responsibilities.

This sketch summarizes the primary AI components and their responsibilities: embeddings and recurrent layers for sequence prediction, and graph population/query methods for Neo4j-backed reasoning.

AI Pipeline: End-to-end (Supplement)

Pipeline Overview

The AI pipeline consists of ingesting behavioral logs, preprocessing into time-ordered sequences, training the sequence model, exporting the best model, synchronizing derived artifacts into the Graph KB (Neo4j), and exposing inference endpoints. Figure 2.18 summarizes this flow.

AI Pipeline E2E (placeholder)

Data Input (data_user500.csv) → Preprocessing → Train (SequenceModel) → Export model_best.pt → Populate Graph (Neo4j) → Serve (FastAPI)

- 1) Ingest CSV / logs
- 2) Feature: product/action embeddings (dims: 16 / 8)
- 3) Train: LSTM (hidden=32), epochs=40 → model_best.pt
- 4) Populate Neo4j: nodes(User,Product), rels(VIEW,CLICK,ADD_TO_CART)
- 5) Serve: FastAPI endpoints (/api/ai/recommend, /api/ai/chat)

Metrics placeholders: sequence_accuracy: xx%, kb_sync_time: xx sec, chat_latency_p95: xx ms

Placeholder — run training and measurement scripts to fill real values.

Figure 2.18: AI pipeline E2E.

I/O examples and run commands

Example CSV row (data_user500.csv) : `'user_id,ts,event_type,product_id'` `'42,2024-10-01T12:05:00Z,VIEW,123'`.

Example inference curl (replace host as appropriate):

```
curl -s -X POST http://localhost:8004/api/ai/recommend \
-H "Content-Type: application/json" \
-d '{"user_id":42,"context":[123,456,789]}'
```

Minimal metrics to capture

Record the following when running experiments; placeholders are acceptable in the thesis and should be replaced with actual numbers when available:

- ‘sequence_accuracy’(testset)-placeholder‘XX%‘
- ‘kb_sync_time’-timetoingestNeventsintoNeo4j-placeholders‘XXs‘
- ‘chat_latency_p95’-95thpercentileresponselatency-placeholders‘XXms‘

Proof-of-Run (Evidence)

Collect minimal, verifiable artifacts and place them under ‘docs/evidence/‘:

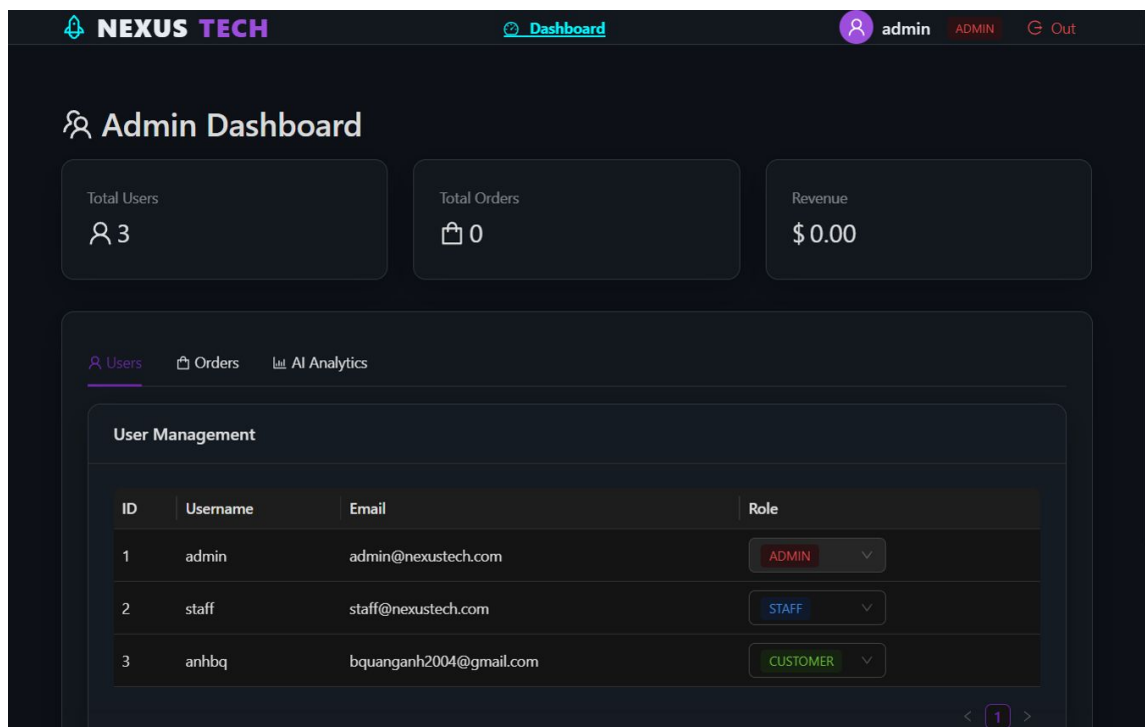


Figure 2.19: Admin dashboard after successful login with role-based access.

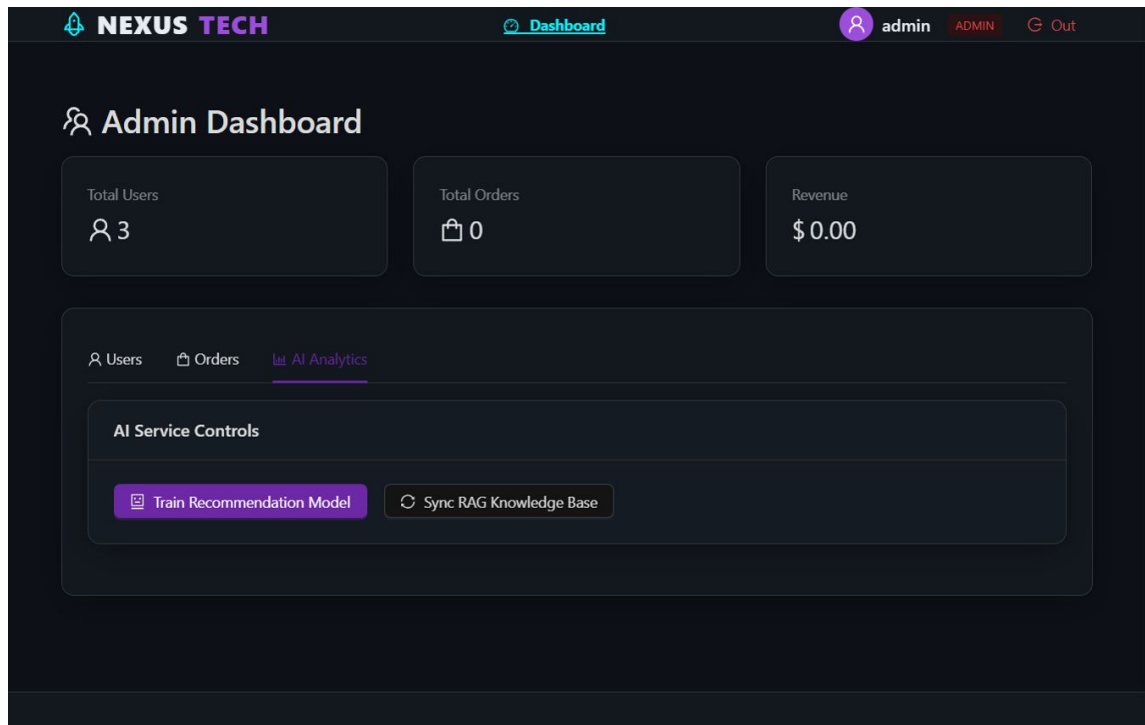


Figure 2.20: Admin AI analytics tab with model training and knowledge sync actions.

1. 'containers.txt': output of 'docker-compose ps' after 'docker-compose up -d'.
2. 'http_traces.json' : *sample request/response pairs for key endpoints (login, create order, ai/recomm...*
2. 'ai_inference.json' : *sample model inference output for a known input.*
2. 'logs.txt': 'docker-compose logs --timestamps' capturing a short successful run.
3. 'screenshot.png': terminal or Overleaf compile screenshot showing success.

Quality, Testing, and Operations (Short)

Test Strategy

- Unit tests per service (models, serializers, utils). Use Django TestCase / pytest.
- Integration tests using the Docker Compose stack or Testcontainers to validate cross-service flows (create order → reduce stock).
- Smoke tests: health endpoints for gateway and services; simple request/response assertions.

Monitoring and Logging

- Centralize logs (file or external aggregator); collect representative logs under ‘docs/evidence/’.
- Export metrics (request counts, latencies) from gateway and AI service; consider Prometheus for collection.

Security (basic)

- JWT: document token expiry and implement refresh token flow.
- Input validation: validate JSON schemas at the gateway and service layers.
- Rate limiting: recommend short-term rate limits at the gateway for unauthenticated endpoints.

2.3 System Design

2.3.1 Individual Service Design

In a microservices architecture, each service operates independently with its own database and application logic. The core services (User, Product, Order) use **Django** and **Django REST Framework (DRF)** with **PostgreSQL**. The AI Service uses **FastAPI** with **SQLite** for behavior logs, graph components (Neo4j), and deep-learning models in PyTorch. This polyglot approach demonstrates a key advantage of microservices.

2.3.2 Service Isolation Strategy

Each service is a separate Django/FastAPI project running in its own Docker container, with its own database:

- `user_db` (PostgreSQL) for the User Service — Port 8001
- `product_db` (PostgreSQL) for the Product Service — Port 8002
- `order_db` (PostgreSQL) for the Order Service — Port 8003
- `behavior.db` (SQLite) + model artifacts (`model_best.pt`) and graph data connectors for the AI Service — Port 8004

This database-per-service pattern ensures that if one service goes down, the others remain operational, and data coupling is strictly avoided. No direct database queries cross service boundaries.

2.3.3 Database Schema and App Structure

Group I: User Service

App Name: users

Django Model:

```
from django.contrib.auth.models import AbstractUser
```

```
class User(AbstractUser):
    ROLE_CHOICES = (
        ('ADMIN', 'Admin'),
        ('STAFF', 'Staff'),
        ('CUSTOMER', 'Customer'),
    )
    role = models.CharField(
        max_length=10, choices=ROLE_CHOICES, default='CUSTOMER'
    )
```

The User model extends Django's `AbstractUser`, inheriting all built-in fields (`username`, `password`, `email`, `first_name`, etc.) and adding a custom `role` field. Authentication uses JWT tokens generated with PyJWT.

Group II: Product Service

App Name: products

Django Models:

```
from django.db import models
```

```
class Category(models.Model):
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True)

class Product(models.Model):
    category = models.ForeignKey(
        Category, related_name='products', on_delete=models.CASCADE
    )
    name = models.CharField(max_length=255)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField(default=0)
    description = models.TextField(blank=True)
```

The `related_name='products'` allows reverse querying from Category to its products. The `description` field stores detailed product information used by the AI Service's RAG knowledge base.

Group III: Order Service

App Name: orders

Django Models:

```
class Order(models.Model):
    STATUS_CHOICES = (
        ('PENDING', 'Pending'),
        ('COMPLETED', 'Completed'),
        ('CANCELLED', 'Cancelled'),
    )
    customer_id = models.IntegerField()
    status = models.CharField(
        max_length=20, choices=STATUS_CHOICES, default='PENDING'
    )
    total_amount = models.DecimalField(
        max_digits=10, decimal_places=2, default=0
    )
    created_at = models.DateTimeField(auto_now_add=True)

class OrderItem(models.Model):
    order = models.ForeignKey(
        Order, related_name='items', on_delete=models.CASCADE
    )
    product_id = models.IntegerField()
    quantity = models.IntegerField(default=1)
    price = models.DecimalField(max_digits=10, decimal_places=2)
```

Note that `customer_id` and `product_id` are stored as simple integers — not foreign keys. The Order Service does not have direct database access to User or Product tables. Instead, it communicates with the Product Service via REST API calls to validate stock and retrieve pricing during order creation.

2.3.4 Connect Services (Inter-service Communication)

Services cannot query each other's databases directly. Instead, they must communicate via well-defined APIs or asynchronous messages.

Synchronous Communication (REST API)

For operations requiring immediate consistency, such as verifying stock availability or fetching up-to-date product pricing during checkout, synchronous HTTP calls are used. For example, when an order is created, the Order Service calls the Product Service to retrieve product details using the `requests` library:

```
import requests
import os

PRODUCT_SERVICE_URL = os.environ.get(
    'PRODUCT_SERVICE_URL', 'http://product_service:8002'
)

def get_product_details(product_id):
    url = f"{PRODUCT_SERVICE_URL}/products/{product_id}/"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    return None
```

Service URLs are configured via environment variables in `docker-compose.yml`, ensuring that services can discover each other by container name within the Docker network.

Asynchronous Communication (REST Callback)

For operations that update state across services after a transaction completes, the system uses synchronous REST callbacks. For example, when a customer places an order in the Order Service, it calls the Product Service's `reduce-stock` endpoint to atomically deduct inventory using Django's F-expressions:

```
# Order Service → Product Service (stock deduction)
requests.patch(
    f"{PRODUCT_SERVICE_URL}/products/{product_id}/reduce-stock/",
    json={"quantity": quantity}
)
```

The Product Service handles this atomically to prevent race conditions:

```
# Product Service - Atomic stock reduction
Product.objects.filter(pk=product.pk).update(
    stock=F('stock') - quantity
)
```

Custom API Gateway Pattern

All client requests are routed through a single entry point — a custom **Django-based API Gateway** running on port 8000. Instead of using a traditional reverse proxy like Nginx, the system implements a lightweight, stateless gateway in Python. This abstracts the internal service topology from the frontend, providing a unified API surface.

The routing logic dynamically resolves paths based on a predefined service map, forwarding HTTP requests using the `requests` library to the corresponding microservices over the internal Docker network:

```
SERVICE_MAP = {  
    'users': 'http://user-service:8001',  
    'products': 'http://product-service:8002',  
    'orders': 'http://order-service:8003',  
    'ai': 'http://ai-service:8004',  
}
```

Requests matching the pattern `/api/<service_name>/<path>` are captured by a centralized proxy view. The gateway carefully filters out hop-by-hop headers (such as `Host`) to ensure compatibility with RFC 1034/1035 and Django's strict validation in downstream services.

This custom gateway pattern offers several advantages: centralized CORS handling, simplified frontend configuration (only one base URL), and the ability to seamlessly inject Python-based middleware for rate limiting, authentication, or logging before requests reach internal services.

Chapter 3

AI Service

This chapter presents the updated design of the standalone **AI Microservice** in this project. Following the current source implementation and AI Service update, the module now emphasizes **sequential behavior learning**, **graph knowledge modeling**, and **Graph-aware conversational support** for E-commerce.

3.1 Architecture Overview

The AI Service is an independent FastAPI-based service (port 8004) with three core capabilities:

1. **Sequence Behavior Prediction** (RNN, LSTM, biLSTM)
2. **Knowledge Base Graph** with Neo4j (User-Product interactions)
3. **GraphRAG-style assistant** using Cypher-based retrieval

The service keeps the same microservice communication style through API Gateway routes (`/api/ai/*`), so the frontend and other services are unaffected by internal AI model changes.

3.2 Module 1: Sequence Behavior Prediction

3.2.1 Dataset and Problem Formulation

The sequence-learning module uses `data_user500.csv` where each user has exactly 8 chronological actions. The training objective is to predict the 8th action from the first 7 steps.

Each timestep is represented by:

- Product embedding (dimension 16)

- Action embedding (dimension 8)

These vectors are concatenated and fed into recurrent architectures.

3.2.2 Model Architecture

```
class SequenceModel(nn.Module):
    def __init__(self, model_type='LSTM', hidden_dim=32):
        super().__init__()
        self.prod_emb = nn.Embedding(num_products, 16)
        self.act_emb = nn.Embedding(num_actions, 8)

        if model_type == 'RNN':
            self.rnn = nn.RNN(24, hidden_dim, batch_first=True)
        elif model_type == 'LSTM':
            self.rnn = nn.LSTM(24, hidden_dim, batch_first=True)
        elif model_type == 'biLSTM':
            self.rnn = nn.LSTM(
                24, hidden_dim // 2,
                batch_first=True,
                bidirectional=True
            )

        self.fc = nn.Linear(hidden_dim, num_actions)

    def forward(self, x):
        emb = torch.cat([self.prod_emb(x[:, :, 0]), self.act_emb(x[:, :, 1])], dim=-1)
        out, _ = self.rnn(emb)
        return self.fc(out[:, -1, :])
```

3.2.3 Training Result and Model Selection

Three recurrent models are trained and compared over 40 epochs:

- **RNN:** Baseline recurrent model with lower stability
- **LSTM:** Best convergence and highest final predictive quality
- **biLSTM:** Competitive, but not better than LSTM in this time-ordered task

The best-performing model is exported as `model_best.pt`.

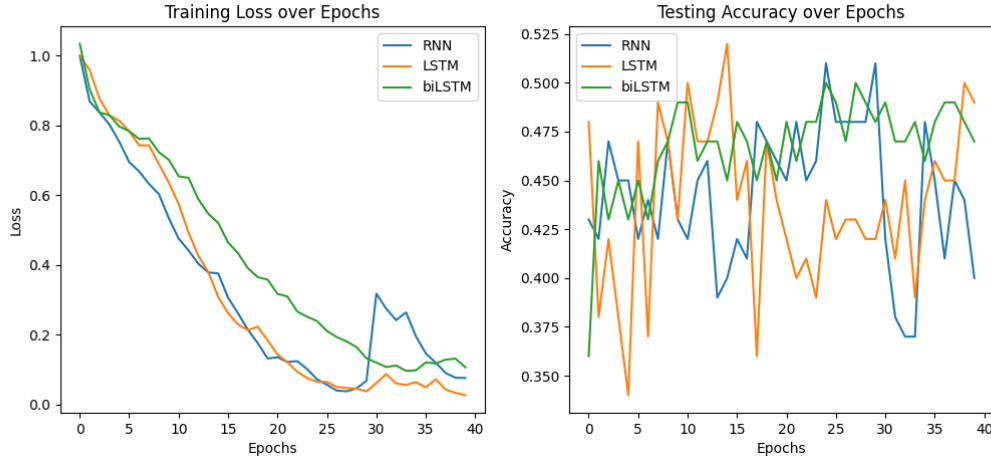


Figure 3.1: Training Loss and Testing Accuracy Comparison for RNN, LSTM, and biLSTM

3.3 Module 2: Knowledge Base Graph with Neo4j

3.3.1 Graph Construction

The graph module maps behavioral records into a knowledge graph:

- Nodes: User, Product
- Relationships: VIEW, CLICK, ADD_TO_CART

```
def populate(self, df):
    self.driver.session().run("""
        UNWIND $users AS u_id MERGE (u:User {id: u_id})
    """, users=df['user_id'].unique().tolist())

    self.driver.session().run("""
        UNWIND $products AS p_id
        MERGE (p:Product {id: p_id, name: 'Product ' + toString(p_id)})
    """, products=df['product_id'].unique().tolist())

    for _, row in df.iterrows():
        self.driver.session().run(f"""
            MATCH (u:User {{id: $uid}})
            MATCH (p:Product {{id: $pid}})
            MERGE (u)-[r:{row['action'].upper()} {{timestamp: $ts}}]->(p)
        """, uid=int(row['user_id']), pid=int(row['product_id']), ts=str(row['timestamp']))
```

3.3.2 Cypher Query Example

```
MATCH (u:User {id: 1})-[r]->(p:Product)
RETURN u, r, p LIMIT 20
```

This representation provides richer behavioral context than flat tabular logs and enables graph-based collaborative reasoning.

3.4 Module 3: GraphRAG Chat and E-commerce Integration

3.4.1 GraphRAG Query Pattern

For trend-intent queries, the chatbot executes Cypher to retrieve top products by cart-add interactions:

```
if "xu huong" in message:
    result = session.run("""
        MATCH (u:User)-[r:ADD_TO_CART]->(p:Product)
        RETURN p.name as Product, Count(r) as TotalAdds
        ORDER BY TotalAdds DESC LIMIT 3
    """)
```

For recommendation intent, a graph collaborative-filtering query is used to find products preferred by behaviorally similar users.

3.4.2 System Integration

The AI module is integrated into the current E-commerce flow through:

- Frontend chat widget for customer-facing AI assistance
- Admin-facing model/knowledge management actions
- API Gateway proxy for all `/api/ai/*` traffic

This keeps deployment and service boundaries stable while allowing AI internals to evolve independently.

3.4.3 AI Components Visual Summary

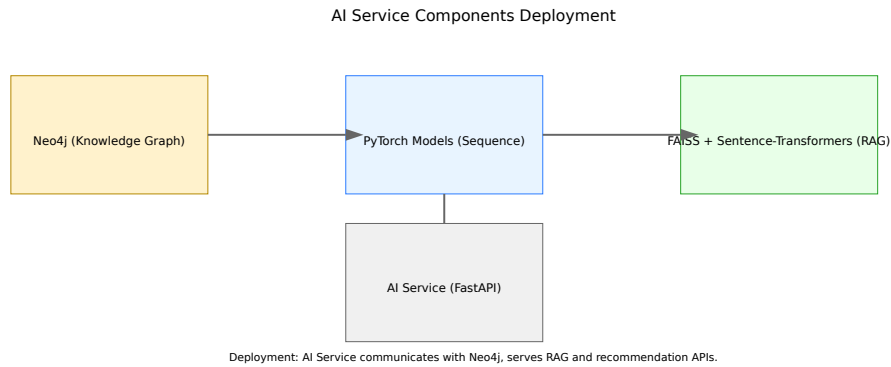


Figure 3.2: AI Service components and deployment overview.

The AI components figure shows three cooperating subsystems: sequence-based behavior models (PyTorch) for short-term intent, a Neo4j knowledge graph for relational reasoning, and a FAISS-backed RAG pipeline for conversational retrieval. The AI Service exposes these capabilities through the gateway under `/api/ai/`.

Chapter 4

Code Implementation and Integration

This chapter presents the core implementation code for the E-commerce Microservices system and the AI Service.

4.1 Project Structure

The entire system is containerized using Docker Compose with 8 containers:

```
src/
docker-compose.yml          # Orchestration for all services
api_gateway/
  manage.py                 # Django-based API Gateway
  api_gateway/              # settings.py, urls.py, wsgi.py
  gateway/views.py          # centralized proxy view
user_service/               # Django (Port 8001)
  users/
    models.py               # User model with roles
    views.py                # Register, Login, UserList, RoleUpdate
    serializers.py          # UserSerializer with password hashing
    management/commands/seed_users.py
product_service/            # Django (Port 8002)
  products/
    models.py               # Category, Product models
    views.py                # CRUD + reduce-stock endpoint
    serializers.py          # CategorySerializer, ProductSerializer
    management/commands/seed_products.py
order_service/              # Django (Port 8003)
  orders/
    models.py               # Order, OrderItem models
```

```

        views.py                # CRUD + top-products + update-status
        serializers.py          # OrderSerializer with stock deduction
ai_service/                    # FastAPI (Port 8004)
    main.py                    # FastAPI app entry point
    routers/                   # analytics, recommendations, chat
    services/                   # behavior, recommendation, rag, graph_rag
    models/                     # ncf model, sequence scripts, database
    data/                       # generated data + Neo4j population scripts
frontend/                      # React + Vite (Port 5173)
    src/
        App.jsx                # Role-based routing
        components/            # Navbar, AdminDashboard, StaffDashboard,
                                # ChatWidget, AIRecommendations, etc.
        contexts/              # AuthContext, CartContext
        services/api.js        # Axios HTTP client

```

4.2 Group II: Product Service Implementation

4.2.1 Django Models

```

from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True)

    def __str__(self):
        return self.name

class Product(models.Model):
    category = models.ForeignKey(
        Category, related_name='products', on_delete=models.CASCADE
    )
    name = models.CharField(max_length=255)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField(default=0)
    description = models.TextField(blank=True)

    def __str__(self):

```

```
return self.name
```

4.2.2 DRF Serializers

```
from rest_framework import serializers
from .models import Category, Product

class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = '__all__'

class ProductSerializer(serializers.ModelSerializer):
    category_name = serializers.ReadOnlyField(source='category.name')

    class Meta:
        model = Product
        fields = '__all__'
```

4.2.3 ViewSets with Stock Reduction

```
from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from django.db.models import F

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    @action(detail=True, methods=['patch'], url_path='reduce-stock')
    def reduce_stock(self, request, pk=None):
        product = self.get_object()
        quantity = request.data.get('quantity', 0)

        if product.stock < quantity:
            return Response(
                {'error': 'Insufficient stock'},
                status=status.HTTP_400_BAD_REQUEST
            )
```

```

# Atomic update using F-expression
Product.objects.filter(pk=product.pk).update(
    stock=F('stock') - quantity
)
product.refresh_from_db()
return Response({
    'product_id': product.id,
    'new_stock': product.stock
})

```

4.3 AI Service: Current Implementation

4.3.1 Recommendation API (NCF)

```

@router.get("/{user_id}")
def recommend(user_id: int, top_k: int = 5):
    return recommendation_service.get_recommendations(user_id, top_k)

@router.post("/train")
def train():
    return recommendation_service.train_model()

```

4.3.2 RAG Chat API (FAISS + Sentence-Transformers)

```

@router.post("/chat")
def chat(req: ChatRequest):
    return rag_service.chat(req.message)

@router.post("/knowledge/sync")
def sync_knowledge():
    return rag_service.sync_knowledge_base()

```

4.3.3 Sequence and Graph Extension Modules

```

# Sequence model training script (offline)
python models/train_sequence_models.py

# Graph KB population script (Neo4j)
python data/populate_neo4j.py

```

These modules are included in the codebase for advanced behavior prediction and graph-based reasoning, while the currently exposed chat endpoint uses the FAISS-based `rag_service`.

4.4 Docker Compose Orchestration

```
services:
  postgres:
    image: postgres:15-alpine
    container_name: ecom_postgres
    environment:
      POSTGRES_USER: root
      POSTGRES_PASSWORD: password
    ports:
      - "5454:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./db-init:/docker-entrypoint-initdb.d
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -U root" ]
      interval: 5s
      timeout: 5s
      retries: 5

  neo4j:
    image: neo4j:5.12
    container_name: ecom_neo4j
    environment:
      - NEO4J_AUTH=neo4j/password
    ports:
      - "7474:7474"
      - "7687:7687"
    volumes:
      - neo4j_data:/data

  user-service:
    build:
      context: ./user_service
    container_name: ecom-user-service
```

```

    depends_on:
      postgres:
        condition: service_healthy
environment:
  - DATABASE_URL=postgres://root:password@postgres:5432/user_db
command: >
  sh -c "python manage.py makemigrations users &&
        python manage.py migrate &&
        python manage.py seed_users || true &&
        python manage.py runserver 0.0.0.0:8001"

product-service:
  build:
    context: ./product_service
  container_name: ecom-product-service
  depends_on:
    postgres:
      condition: service_healthy
environment:
  - DATABASE_URL=postgres://root:password@postgres:5432/product_db
command: >
  sh -c "python manage.py makemigrations products &&
        python manage.py migrate &&
        python manage.py seed_products || true &&
        python manage.py runserver 0.0.0.0:8002"

order-service:
  build:
    context: ./order_service
  container_name: ecom-order-service
  depends_on:
    postgres:
      condition: service_healthy
environment:
  - DATABASE_URL=postgres://root:password@postgres:5432/order_db
  - PRODUCT_SERVICE_URL=http://product-service:8002
command: >
  sh -c "python manage.py makemigrations orders &&
        python manage.py migrate &&"

```

```
python manage.py runserver 0.0.0.0:8003"
```

```
ai-service:
```

```
  build:
```

```
    context: ./ai_service
```

```
  container_name: ecom-ai-service
```

```
environment:
```

```
  - PRODUCT_SERVICE_URL=http://product-service:8002
```

```
  - ORDER_SERVICE_URL=http://order-service:8003
```

```
  - NEO4J_URI=bolt://neo4j:7687
```

```
  - NEO4J_USER=neo4j
```

```
  - NEO4J_PASSWORD=password
```

```
depends_on:
```

```
  - product-service
```

```
  - order-service
```

```
  - neo4j
```

```
command: uvicorn main:app --host 0.0.0.0 --port 8004
```

```
api-gateway:
```

```
  build:
```

```
    context: ./api_gateway
```

```
  container_name: ecom-api-gateway
```

```
depends_on:
```

```
  - user-service
```

```
  - product-service
```

```
  - order-service
```

```
  - ai-service
```

```
ports:
```

```
  - "8000:8000"
```

```
frontend:
```

```
  build:
```

```
    context: ./frontend
```

```
  container_name: ecom-frontend
```

```
ports:
```

```
  - "5173:5173"
```

```
environment:
```

```
  - VITE_API_URL=http://localhost:8000/api
```

```
command: npm run dev -- --host
```

```
volumes:
  postgres_data:
  neo4j_data:
```

4.5 Chapter 5 — Demo and Code Runbook

4.5.1 Run and Demo Steps

Below are the commands used to build and run the full demo locally (assumes Docker and Docker Compose installed):

```
# Build and start all services (from src/)
docker compose up --build
```

```
# (Optional) Recreate only AI service after model changes
docker compose up -d --no-deps --build ai-service
```

```
# Train NCF model (inside ai_service container or locally)
python src/ai_service/services/recommendation_service.py # or use exposed POST /recom
```

```
# Sync RAG KB
curl -X POST http://localhost:8004/knowledge/sync
```

```
# Populate Neo4j (if running locally)
python src/ai_service/data/populate_neo4j.py
```

4.5.2 Code Highlights (paths)

- API endpoint: `src/ai_service/main.py` — *RecommendationAPI* : `src/ai_service/routers/recommen`

4.5.3 Demo Checklist

- Build images and bring containers up via Docker Compose
- Visit API Gateway at `http://localhost:8000/api` to test proxied endpoints
- Trigger NCF training via `POST /api/ai/recommendations/train`
- Sync RAG knowledge via `POST /api/ai/knowledge/sync`
- Test chat via `POST /api/ai/chat` with sample messages

4.5.4 Minimal Example: Calling Recommendation API

```
curl http://localhost:8000/api/ai/recommendations/1?top_k=5
```

4.6 Verification Results

The following tests were conducted to verify the system's correctness:

Test Case	API	Result
Stock deduction (50 $\rightarrow$ 45)	POST /orders/ + GET /products/1/	PASS
Admin login	POST /users/login/ (admin)	PASS
Staff login	POST /users/login/ (staff)	PASS
Behavior tracking	POST /ai/analytics/track	PASS
NCF model training	POST /ai/recommendations/train	PASS
Product recommendations	GET /ai/recommendations/1	PASS
Knowledge base sync	POST /ai/knowledge/sync	PASS
RAG chat response	POST /ai/chat	PASS
Sequence data generation (500 users)	python data/generate_seq_data.py	PASS
RNN/LSTM/biLSTM training	python models/train_sequence_models.py	PASS
Neo4j graph population	python data/populate_neo4j.py	PASS
Order status update	PATCH /orders/4/update-status/	PASS

Table 4.1: System Integration Test Results

Conclusion

This project successfully designed and implemented a complete E-commerce platform using Microservices Architecture and AI Integration. The key accomplishments include:

1. **Microservices Decomposition:** The system was decomposed into 4 independent services (User, Product, Order, AI) using Domain-Driven Design principles, each with its own database and technology stack.
2. **Inter-service Communication:** Services communicate exclusively via REST APIs through a Django-based API Gateway, ensuring loose coupling and independent deployability.
3. **AI-Powered Intelligence:** The AI Service demonstrates three cutting-edge capabilities:
 - Sequential behavior prediction with RNN, LSTM, and biLSTM models
 - Graph-based knowledge representation using Neo4j and Cypher
 - GraphRAG-style conversational support for trend and recommendation queries
4. **Full-stack Implementation:** The frontend (React) provides role-based dashboards for Admin, Staff, and Customer, with integrated AI features including a floating chatbot and personalized recommendations.
5. **Containerized Deployment:** The entire system runs as 8 Docker containers orchestrated by Docker Compose, demonstrating production-ready deployment practices.

The project demonstrates that combining Microservices Architecture with modern AI techniques creates a scalable, maintainable, and intelligent e-commerce platform suitable for real-world applications.